Documentation of Corrective, Preventive, and Improvement Actions for the Software

Vehicle Washing Management System (Vehicle Washing)

Management Procedure, Action Register, Root Cause Analysis, Follow-Up, and Lessons Learned

Brayan Estiven Patiño Cabrera (Project Leader), Juan Camilo Guilombo Osorio (Frontend Leader), and Merari Urbano (Backend Leader)

Technologist in Software Analysis and Development (ADSO), Servicio Nacional de Aprendizaje (SENA), Training Center: Industria

Ficha 3145556

Instructor: Karol Daniela Correa

October 16, 2026

Document Type: Register of Corrective, Preventive, and Improvement Actions (Development Team and Instructor) · Version 2.0

Version Control

Table 1 records the change history of this document. The action register (sections 9 to 12) is a living document: each time an action is opened, updated, or closed, the row of the action is modified and, at least once per sprint, a row is added to this table.

Table 1

Version History of the Actions Document

Version	Date	Description of the change	Responsible
1.0	2026-10-09	Creation of the document (in Spanish and English). The management procedure is defined, and 23 closed corrective actions, 13 open corrective actions, 14 preventive actions, and 13 improvement actions are recorded, verified against the code of the eleven repositories (*develop* branch as of October 9, 2026), the Git history, the risk register, and the technical backlog of the project.	Development team
2.0	2026-10-16	Review against the code of October 16, 2026. Eleven open corrective actions were closed (ACO-01 to ACO-03 and ACO-05 to ACO-12, now AC-24 to AC-34) and five improvement actions (AM-01, AM-04, AM-08, AM-10, and AM-11); five nonconformities found in the complete review of the system were recorded and closed (AC-35 to AC-39); the preventive actions, indicators, lessons learned, and traceability were updated.	Development team

Note. Dates are written in the ISO 8601 format (year-month-day). Prepared by the development team.

Table of Contents

- Version Control
- 4. Introduction
  - 4.1 Reference Framework
  - 4.2 Sources of This Document
- 5. Objective
  - 5.1 General Objective
  - 5.2 Specific Objectives
- 6. Scope
- 7. Definitions and Classification
  - 7.1 Types of Action
  - 7.2 Severity and Priority
  - 7.3 States of an Action
- 8. Action Management Procedure
  - 8.1 Identification Sources
  - 8.2 Stages of the Procedure
  - 8.3 Root Cause Analysis
  - 8.4 Effectiveness Verification
  - 8.5 Roles
- 9. Register of Closed Corrective Actions
- 10. Register of Open Corrective Actions

- 11. Register of Preventive Actions
- 12. Register of Improvement Actions
- 13. Follow-Up Indicators
- 14. Lessons Learned
- 15. Glossary
- References
- 16. Annexes
    - Annex A. Action Record Form
    - Annex B. Root Cause Analysis Example
    - Annex C. Traceability Between Actions, Risks, and Technical Debt

4. Introduction

The Vehicle Washing Management System (Vehicle Washing), internally called *LavaRapido*, is a platform for bookings, operator assignment, execution and rating of services, verification of payments by QR code, and loyalty points for a car wash. It consists of a gateway, six microservices (five in Java 21 with Spring Boot 4 and one in .NET 8), a SQL Server 2022 database, RabbitMQ, and two clients, web (Angular 21) and mobile (React Native with Expo) (Vehicle Washing Development Team, 2026a).

During its construction, its reviews with the instructor, and the writing of its manuals, defects, risks, and improvement opportunities have been detected. This document establishes **how** they are managed (identification, recording, root cause analysis, planning, execution, verification, and closure) and **records** the actions taken and pending, so that any team member, the instructor, or a future maintenance team knows what was corrected, why, with what evidence, and what remains to be done.

4.1 Reference Framework

- **ISO/IEC/IEEE 12207:2017.** The Maintenance process (6.4.13) includes correcting defects, preventing failures, and improving the software in operation, and the Quality Assurance process (6.3.8) requires treating the incidents and problems detected and verifying that the actions are effective (International Organization for Standardization [ISO] et al., 2017).
- **ISO/IEC/IEEE 14764:2022.** It elaborates the maintenance process of ISO/IEC/IEEE 12207 and defines the types of maintenance; this document uses the corrective (correct discovered defects), preventive (correct latent defects before they become failures), adaptive (adjust the software to changes in the environment), and perfective (improve the software) types (ISO et al., 2022).
- **ISO 9001:2015.** Clause 10.2 requires reacting to a nonconformity, determining its causes, implementing the necessary action, reviewing its effectiveness, and retaining documented information; clause 10.3 requires continual improvement. The standard replaced "preventive action" as a separate requirement with risk-based thinking (clause 6.1), so in this document preventive actions arise from the risk register (ISO, 2015).

- **PDCA cycle.** Each action goes through the plan, do, check, and act cycle, which is the basis of continual improvement in ISO 9001.

4.2 Sources of This Document

The actions were verified against the code of the eleven repositories (*develop* branch as of October 16, 2026) and their Git history, and against the risk register, the technical backlog, the incident management, the testing strategy, and the architecture decision records (ADRs) of the project (Vehicle Washing Development Team, 2026a). The findings of the review of the manuals of October 7, 9, and 16, 2026, and of the complete review of the system of October 16 (Annex B of the Technical Manual and the annexes of the other manuals) are also included.

5. Objective

5.1 General Objective

To document the procedure and the register of the corrective, preventive, and improvement actions of the Vehicle Washing Management System, to ensure that each defect, risk, or improvement opportunity is treated in a traceable and verifiable way, with learning for the team.

5.2 Specific Objectives

- Define the types of action, their classification by severity and priority, and their life cycle of states.
- Establish a single procedure to identify, record, analyze, execute, verify, and close actions.
- Record the corrective actions already performed with their root cause and their evidence.
- Record the pending corrective, preventive, and improvement actions with their person responsible, priority, and closure criterion.
- Define indicators to measure the effectiveness of the procedure and consolidate the lessons learned.

6. Scope

It covers: the six backend services, the gateway, the database, the Docker infrastructure, the web application, the mobile application, and the project documentation (manuals, ADRs, contracts, and data models), from construction to operation after implementation.

It does not cover: the management of new product requirements (user stories of the product backlog), except when an improvement need of the software already built is recorded as an improvement action; nor the academic management of the project (grades, program deliveries).

7. Definitions and Classification

7.1 Types of Action

Table 2

Types of Action and Their Relationship With the Types of Maintenance

Type	Definition	Trigger	Type of maintenance (ISO/IEC/IEEE 14764)	Prefix
Corrective action	Eliminates the cause of a nonconformity that **has already occurred** (defect, failure to meet a requirement, an ADR, or the documentation) so that it does not recur.	Incident, failed test, review, remark of the instructor or of acceptance.	Corrective	AC (closed) and ACO (open)
Preventive action	Eliminates the cause of a **potential** nonconformity before it occurs.	Risk of the risk register, technical debt, trend analysis.	Preventive and adaptive	AP
Improvement action	Increases the value, quality, or efficiency of the software without an existing failure.	Retrospective, user suggestion, recommended evolution of the architecture.	Perfective and adaptive	AM

Note. Prepared by the authors from ISO (2015) and ISO et al. (2022). The prefixes keep the Spanish initials so that both versions of the register share the same identifiers.

A **correction** (for example, restarting a service or editing a record) is not a corrective action: it resolves the symptom, but not the cause. In the project's incident management, the correction corresponds to the "mitigate" step; the corrective action is the change of code, configuration, or process that prevents the incident from recurring.

7.2 Severity and Priority

Severity uses the same scale as the project's incident management (Table 3). Priority combines severity with the probability of occurrence and the effort, and is expressed as High, Medium, or Low, as in the technical backlog.

Table 3

Severity Scale

Level	Definition	Example	Response time
P0	Nothing works.	SQL Server down; no service starts; login impossible.	Immediate.
P1	A core flow is broken.	A booking, an assignment, the start of a service, or the report or approval of a payment is impossible.	The same day.
P2	A secondary function fails and a workaround exists.	Emails do not arrive, but the internal inbox works.	Within the sprint.
P3	Cosmetic or documentary.	Untranslated text, wrong icon, outdated manual.	Backlog.

Note. Taken from the incident management of the project (Vehicle Washing Development Team, 2026a).

7.3 States of an Action

Each action goes through the states **Open** (recorded), **Under analysis** (the root cause is being sought), **Planned** (it has a person responsible, a solution, and a date), **In progress** (branch and pull request under way), **Under verification** (deployed; its effectiveness is being checked), and **Closed** (effective and with evidence). An action whose verification fails returns to *Under analysis*. An action that the team decides not to perform is marked **Accepted as a risk**, with the justification and the approval of the project leader.

8. Action Management Procedure

8.1 Identification Sources

- Incidents of the project's incident management and their post-mortems (P0 and P1).
- Failed automated tests (*./mvnw verify*, *dotnet test*, *ng build*) and manual end-to-end tests before each review.

- Code reviews (pull requests) and reviews by the instructor.
- Smoke and acceptance tests of the Implementation Procedure.
- Comparison between the code, the ADRs, and the manuals (discrepancy annexes).
- Risk register and technical backlog (source of preventive actions).
- Retrospectives of each sprint and user comments (source of improvement actions).

8.2 Stages of the Procedure

Table 4

Stages of the Action Management Procedure

Stage (PDCA)	Activity	Responsible	Output
1. Identify (Plan)	Detect the nonconformity, risk, or opportunity and report it in the team chat.	Whoever detects it	Notice with description and evidence.
2. Record (Plan)	Create the row in the corresponding register (sections 9 to 12) with the form of Annex A; assign ID, type, severity, and person responsible.	Leader of the affected area	Action in the Open state.
3. Contain (Do)	Only for P0 and P1 corrective actions: apply the immediate correction (restart, adjust *.env*, start the missing container).	Service owner	Flow restored.
4. Analyze the cause (Plan)	Apply the five whys or the cause-and-effect diagram (section 8.3).	Service owner	Documented root cause.
5. Plan (Plan)	Define the solution, the closure criterion, the effort, and the date; if a technical decision changes, create or modify an ADR.	Person responsible with the project leader	Action in the Planned state.
6. Execute (Do)	*fix/** or *feat/** branch, change with tests that reproduce the defect, pull request reviewed by another person, and documentation updated in the same pull request (definition of done).	Person responsible	Pull request merged into *develop*.
7. Verify effectiveness (Check)	Check the closure criterion in the implemented environment and that the problem does not reappear (section 8.4).	Person other than the executor	Verification evidence.
8. Close and learn (Act)	Mark Closed, record the lesson learned (section 14), and update the risk register and the technical backlog.	Project leader	Action in the Closed state.

Note. Prepared by the authors from clause 10.2 of ISO (2015) and the definition of done of the project.

8.3 Root Cause Analysis

For corrective actions of severity P0, P1, and P2, documenting the root cause is mandatory. Two techniques are used:

- **Five whys:** the question "why did it happen?" is asked about each answer until reaching a cause on which the team can act (process, design, configuration, or knowledge). Annex B contains a real example from the project.
- **Cause-and-effect (Ishikawa) diagram:** when there are several possible causes, they are classified into the categories *code*, *configuration*, *data*, *infrastructure*, *process*, and *people* before choosing the main cause.

The root cause is **never** recorded as "human error": if a person made a mistake, the question is what in the process, the tool, or the documentation allowed the mistake (the blameless approach of the project's post-mortems).

8.4 Effectiveness Verification

An action is closed only when its closure criterion is met and there is verifiable evidence: a commit or pull request, an automated test that reproduces the defect and now passes, the result of a manual test, a database query, or a screenshot.

For P0 and P1 corrective actions, it is also required that the problem does not reappear in the next review or implementation. Verification is performed by a person other than the one who executed the action.

8.5 Roles

Table 5

Roles in Action Management

Role	Person	Responsibility
Project leader	Brayan Estiven Patiño Cabrera	Approves priorities, accepts risks, verifies closure, and maintains this document.
Backend leader	Merari Urbano	Responsible for the actions of services, database, and infrastructure.
Frontend leader	Juan Camilo Guilombo Osorio	Responsible for the actions of the web and mobile applications.
Author of each pull request	Any team member	Updates the affected documentation in the same pull request.
Instructor	Karol Daniela Correa	Source of remarks; reviews the document at the deliveries.

Note. Prepared by the authors. Each service also has an owner defined in the project's service catalog.

9. Register of Closed Corrective Actions

Table 6 records the nonconformities already corrected, with their root cause and the closure evidence. The "Origin" column refers to the closed risk (C-), the technical debt item (TD-), or the corresponding ADR.

Table 6

Closed Corrective Actions

ID	Nonconformity	Sev.	Root cause	Action performed	Closure evidence	Origin	Closed
AC-01	Booking times appeared shifted by five hours.	P1	The JVM converted dates with the machine's time zone instead of the business's.	The booking-service JVM runs in UTC; dates are stored in UTC and converted only at the edges with *BOOKING_ZONE=America/Bogota*.	*AvailabilityPolicyTest* tests and correct bookings in the end-to-end test.	C-001	2026-10-04
AC-02	Services failed at startup after *docker compose up*.	P0	The services tried to connect before SQL Server was healthy.	Health checks on *sqlserver* and *rabbitmq* and *service_healthy* and *service_completed_successfully* dependencies in the Compose file.	*docker-compose.yml* with conditional *depends_on*.	C-002	2026-10-04
AC-03	Two different implementations of the booking service existed.	P1	Coding started without an agreed data model.	booking-service was rebuilt from the agreed SQL model; team rule: agree on the model before coding a service.	A single repository *lavarapido-booking-service* aligned with *06-data/models.md*.	C-003	2026-10-04
AC-04	With *--profile app*, every payment route failed.	P1	payment-service had no Dockerfile or Compose service, and the gateway container did not receive *ROUTE_PAYMENT_URL*.	Multi-stage .NET 8 Dockerfile, *payment-service* service in Compose, and *ROUTE_PAYMENT_URL* in the gateway.	*lavarapido-infra* commit of 2026-10-08 ("wire payment-service to booking-service and rabbitmq").	TD-014	2026-10-08
AC-05	*BOOKING_SERVICE_URL* was missing for payment-service in Compose; crediting points failed in containers.	P1	The variable was omitted when the service was created (pre-existing gap found in ADR-015).	Variable added with *http://booking-service:3003/api/v1*.	Current *docker-compose.yml*.	ADR-015	2026-10-08
AC-06	The customer was not notified when a payment was approved, rejected, or refunded.	P2	payment-service raised domain events but had no RabbitMQ publisher.	Publisher with *RabbitMQ.Client* and events *payment.confirmed*, *payment.rejected*, and *payment.refunded*, with an email to the customer.	ADR-011 section 9; *PAYMENT_** types in notification-service (migration 028).	TD-015	2026-10-08
AC-07	The same transaction reference could pay two bookings.	P1	The uniqueness of the reference was not validated when a payment was reported (control of ADR-008 and RF-016).	Validation in payment-service with the error code *TRANSACTION_REFERENCE_REUSED*.	*ErrorCodes.cs* of payment-service.	TD-016	2026-10-08
AC-08	payment-service only tested its domain; the role and conflict (409) rules were unprotected.	P2	Application and API tests were postponed when the service was built.	Projects *PaymentService.Application.UnitTests* and *Infrastructure.IntegrationTests*.	*tests/* folder with command, query, promotion, and persistence tests.	TD-018	2026-10-08

ID	Nonconformity	Sev.	Root cause	Action performed	Closure evidence	Origin	Closed
AC-09	Points earned by a booking were not reversed when its payment was refunded, and two simultaneous movements could leave a wrong balance.	P1	The points ledger had no reversal movement and no concurrency control.	*REVERSED* movement type and *sequence_no* column unique per customer (migration *pay-020*); the second concurrent movement answers 409.	Migration *020-loyalty-reversal-and-sequence.sql*.	ADR-015	2026-10-08
AC-10	Payment approval compared the booking total with itself and accepted invalid images.	P1	The amount declared by the customer was neither sent nor compared.	The paid amount is sent and compared with the amount due in the review window, and images are validated as PNG or JPG.	payment-service and web commits of 2026-10-08.	Manuals v3.0	2026-10-08
AC-11	A "custom roles" catalog existed with no effect on authorization, contradicting ADR-010.	P2	A feature was built without checking it against the recorded decisions.	*security.custom_role* was removed (migration 026); the three roles stay fixed and the permission matrix of *security.role_permission* is edited.	ADR-015 section 2; endpoint */api/v1/admin/roles-permissions*.	ADR-015	2026-10-08
AC-12	A fixed operator-per-bay assignment had been built, contrary to ADR-010.	P2	Same as AC-11.	The *service_bay.operator_id* column was reverted (migration 024); the bay is chosen per booking and the operator per execution.	ADR-015 section 1.	ADR-015	2026-10-08
AC-13	Promotions showed a package price that was never applied to the charge.	P2	The promotions model was used for display only.	Coupons with a real percentage discount, redeemable with points (*POST /api/v1/loyalty/redeem*), and a form without price, duration, or status.	ADR-015 sections 3 and 5.	ADR-015	2026-10-09
AC-14	When a service was edited, its loyalty points went back to 0.	P2	booking-service binds the field to a primitive *int* and the clients omitted the field.	The web and the app always send *loyaltyPoints*; the app keeps the price of each vehicle type when editing.	ADR-015 section 5; mobile app commits of 2026-10-09.	ADR-015	2026-10-09
AC-15	"Pay" was offered on bookings already paid or under review.	P2	Each screen repeated the payment rule instead of asking payment-service.	Endpoint *GET /api/v1/payments/me/bookings* with the *payable* flag; the web and the app show the button only with that flag.	ADR-015 section 5.	ADR-015	2026-10-09
AC-16	The notification switches were saved only in the browser or the phone and had no effect.	P2	There were no columns for those preferences in the account.	Columns in *security.user_preference* (migration 030), partial update of preferences, and application of the switches in notification-service.	ADR-011 section 12.	ADR-011	2026-10-09
AC-17	Theme and language were stored per browser, and a second account on the same device inherited them.	P2	Preferences were read from *localStorage* without querying the account.	Load after login and save with *GET/PUT /api/v1/users/me/preferences*; palettes accepted by security-service (migration 020).	*core/services/preferences.ts* of the web.	TD-002	2026-10-09
AC-18	The administrator could send any notification type by hand (for example, a fake "payment approved").	P1	The manual endpoint did not restrict the type.	Only *SYSTEM_MESSAGE* and *INSPECTION_REPORT* are accepted; any other type answers 400 *NOTIFICATION_TYPE_NOT_ALLOWED*.	ADR-011 section 11.	Security review	2026-10-09
AC-19	Null or nonexistent identifiers were accepted when updating the permissions of a role.	P2	Input validation was missing in the use case.	Rejection with 400 *UNKNOWN_PERMISSION* and a translated message in the web.	security-service commit of 2026-10-08.	ADR-015	2026-10-08
AC-20	The inspection report was shared by WhatsApp, outside the system's channels.	P3	Earlier product decision.	"Notify customer" button that sends the summary by notification and email (*INSPECTION_REPORT*); WhatsApp was removed from the business data and the help center.	ADR-011 section 11; web commits of 2026-10-09.	ADR-011	2026-10-09
AC-21	The app opened in the browser with Expo (port 8081) was rejected by CORS.	P2	Origin 8081 was not in the default values.	The default *CORS_ALLOWED_ORIGINS* includes *http://localhost:8081* in every service.	*lavarapido-infra* and gateway commits.	—	2026-10-09
AC-22	The old operator tasks route showed an empty page.	P3	Route without a component after a reorganization.	Redirect to "Assigned services" and removal of the empty page.	Web commit of 2026-10-09.	—	2026-10-09
AC-23	Manuals v3.0 became outdated by the changes of October 8 and 9 (points, coupons, refunds, roles, preferences, payments in Docker), existed only in English, and the Installation Manual said to clone the operations repository with a name that breaks the build.	P3	The documentation was not updated in the same pull requests (risk R-010).	Manuals updated to version 3.1 in Spanish and English, and creation of the Implementation Procedure and of this document.	Version control of each manual (2026-10-09).	R-010	2026-10-09
AC-24	The web test suite did not compile (formerly ACO-01).	P2	Six scaffolding *.spec.ts* files pointed to classes that no longer existed.	The test files were fixed and tests for the API error messages were added.	*npx ng test --watch=false*: 43 tests in 37 files passed (2026-10-16).	TD-001	2026-10-16

ID	Nonconformity	Sev.	Root cause	Action performed	Closure evidence	Origin	Closed
AC-25	There was no login attempt limit (formerly ACO-02).	P1	RF-004b had not been implemented.	*LoginLockout*: five failed attempts within 15 minutes lock the account for 15 minutes (423 ACCOUNT_LOCKED); a successful login or a password reset clears it.	Changelog 034, *AuthenticationServiceTest* and *ApiSecurityWebTest*; message shown on the web and mobile.	TD-003, RF-004b	2026-10-16
AC-26	The acceptance of the Terms and of the Data Processing Policy was not stored on the server (formerly ACO-03).	P1	The consent table was left pending in ADR-010.	*security.app_user* stores the date and time each document was accepted (*terms_accepted_at*, *data_policy_accepted_at*); web and mobile registration sends *acceptsTerms* and *acceptsDataPolicy*.	Changelog 039, 147 security-service tests passed, checkboxes in both clients.	Law 1581 of 2012	2026-10-16
AC-27	The payment-service health answered *UP* with the database down, and the service did not handle *X-Correlation-Id* (formerly ACO-05).	P2	The endpoint returned a fixed value.	*/health* and */actuator/health* query SQL Server with a 3-second limit; *CorrelationIdMiddleware* takes or creates the identifier and forwards it to booking-service.	With the database stopped the service answers 503 DOWN; 141 payment-service tests passed.	TD-017	2026-10-16
AC-28	The refund texts were fixed in Spanish (formerly ACO-06).	P3	They were left in a temporary constants file.	Texts moved to the language files in Spanish, English, French, and Portuguese; *payment-texts.ts* was deleted.	Administrator payments screen in the four languages.	Administrator Manual	2026-10-16
AC-29	The operator's "Report progress" button said it notified the customer but sent nothing (formerly ACO-07).	P3	The function was left with only the visual confirmation.	The button was removed on the web and mobile; the customer receives the real start and finish notices.	Assigned services screen without the button.	User Manual	2026-10-16
AC-30	The landing page announced a service "wherever you are", and the operator history could show payment methods the system does not support (formerly ACO-08).	P3	Mockup texts.	The home page describes the service at the car wash with the real establishment data; the operator history no longer shows card or PSE.	Home page in the four languages.	Technical Manual, Annex B	2026-10-16
AC-31	An unknown address showed an empty page (formerly ACO-09).	P3	The wildcard route was missing.	404 page (*features/not-found*, route **) with a link to the home page.	A nonexistent address shows the 404 page.	TD-009	2026-10-16
AC-32	The inspection report and its photos were kept only in the browser (formerly ACO-10).	P2	There was no aggregate in the backend.	*InspectionReport* aggregate in operations-service with phases, findings, and photos (changelog 033); the public link opens on any device and the notice to the customer includes it.	Test *inspection-flow.test.js* of *lavarapido-infra/e2e* passed.	RF-036	2026-10-16
AC-33	The business data of Settings were kept only in the browser (formerly ACO-11).	P3	The extension of *booking.establishment* waited for the definition of its fields.	Changelog 032 and *GET/PUT /api/v1/admin/establishment*; the home page and the help center read those data.	Changes are visible from another account and on the public page.	TD-013, RF-037	2026-10-16
AC-34	The *lavarapido-infra* README and the team guide described services and screens that already existed as pending (formerly ACO-12).	P3	Documents not updated when the services were integrated.	Both documents updated.	Commits of 2026-10-16 in *lavarapido-infra*.	R-010	2026-10-16
AC-35	Two simultaneous requests could try to create the same operator, and one failed with error 500.	P2	The operator creation read and then inserted without handling the race between requests.	The insert runs in its own transaction and, if the row already exists, it is read again instead of failing.	*OperatorPersistenceAdapterTest*; 54 operations-service tests passed.	Review of 2026-10-16	2026-10-16
AC-36	When a time outside the schedule or already past was requested, the system answered "That time was just taken".	P3	The 409 response did not distinguish the reason.	The response carries *reason* (CLOSED_DAY, OUTSIDE_HOURS, TIME_PASSED, or NO_FREE_BAY), and the web and mobile show a message for each case.	Test *access-rules.test.js* (*reason: OUTSIDE_HOURS*) and *api-error.spec.ts*.	Review of 2026-10-16	2026-10-16
AC-37	The "Delete account" option said it erased the account and all the data, but it only deactivates it.	P2	Text inherited from the mockup, contrary to what the server does.	The option is called "Deactivate my account" and explains that the history is kept, on the web and mobile and in the four languages.	Web and mobile profile; User Manual 4.0.	Law 1581 of 2012	2026-10-16
AC-38	The manuals and project documents became outdated with respect to the code of October 16 (lockout, automatic assignment, inspection, 57 tables), and the figures were placeholders.	P3	Changes integrated without updating the documentation in the same pull request (R-010).	Manuals 4.0 with real screenshots, Implementation Procedure 2.0, this document 2.0, and the project's technical documentation reviewed against the code.	Version control of each document (2026-10-16).	R-010	2026-10-16

ID	Nonconformity	Sev.	Root cause	Action performed	Closure evidence	Origin	Closed
AC-39	With two accounts of the same method (for example, two Nequi), the web payment screen marked both at once and always showed the QR of the first one.	P3	The chosen option was stored by method type and not by account.	The selection is stored by account and, if two accounts share a method, each option shows its holder.	*payment.spec.ts* (two Nequi accounts); 43 web tests passed.	Test Documentation, defect 6	2026-10-16

Note. Prepared by the authors from the Git history of the repositories, the risk register, and the ADRs of the project (Vehicle Washing Development Team, 2026a).

10. Register of Open Corrective Actions

Table 7

Open Corrective Actions

ID	Nonconformity	Sev.	Root cause	Proposed action	Responsible	Priority	Closure criterion
ACO-04	The table *audit.audit_log* exists but nothing writes to it.	P2	The triggers were not created and *SESSION_CONTEXT(N'user_id')* is not set.	Audit triggers on the sensitive tables and session context in each service, or remove the table from the design through an ADR.	Backend leader	Medium	Changes of prices, payment accounts, and roles visible in the audit trail.
ACO-13	Entities edited by administrators do not use optimistic locking: in concurrent edits, the last write wins.	P3	The *row_version* column exists but is not mapped with *@Version*.	Map *@Version* in booking, customer, and security and answer 409 on conflict.	Backend leader	Low	Test of two updates with the same version. (TD-008)

Note. Prepared by the authors from the technical backlog, Annex B of the Technical Manual, and the verification of the code as of October 16, 2026. Actions ACO-01 to ACO-03 and ACO-05 to ACO-12 were closed and moved to Table 6 (AC-24 to AC-34).

11. Register of Preventive Actions

Preventive actions address the causes of risks that have not yet materialized or that could recur.

Table 8

Preventive Actions

ID	Risk or potential cause	Preventive action	Responsible	Priority	Status	Origin
AP-01	Regressions reach *develop* unnoticed.	Jenkins pipeline per repository (*./mvnw -B verify*, *dotnet test*, *ng build*, and *ng test*) and the rule of merging only with a green pipeline.	Project leader	High	Open	R-003, TD-006
AP-02	An event is lost between the commit and the publication.	Outbox pattern in security, booking, operations, and payment.	Backend leader	Low	Accepted as a risk until leaving the academic scope	R-005, TD-011
AP-03	Secret leakage (*.env*, mail app password, *JWT_SECRET*).	Keep *.env* ignored, check with *git check-ignore* before each implementation, and rotate secrets every quarter or on suspicion.	Whole team	High	In progress (permanent)	R-004

ID	Risk or potential cause	Preventive action	Responsible	Priority	Status	Origin
AP-04	Data loss before a demonstration or in operation.	Automated daily backup with verification and a monthly restoration test (Administrator Manual, section 14).	IT administrator	High	Open	R-008
AP-05	The documentation drifts from the code again.	Pull request checklist that requires updating the affected models, OpenAPI contracts, ADRs, and manuals; review of the manuals before each implementation.	Author of each pull request	Medium	In progress (permanent)	R-010
AP-06	Implementing with *MESSAGING_ENABLED=false*, without *INTERNAL_API_KEY*, or with the *dev* profile.	Checklist of Annex A of the Implementation Procedure.	IT administrator	Medium	Closed (included in the procedure)	R-I02, R-I03
AP-07	Persistence defects (locks and triggers) seen only by hand.	Persistence tests with Testcontainers and a real SQL Server in booking-service.	Backend leader	Low	In progress (the end-to-end suite covers part of the risk)	TD-010
AP-08	All services use the *sa* account.	One login per service with permissions only on its schemas and a separate account for the migrations.	Backend leader	Medium	Open	Security policy
AP-09	A user action cannot be followed across services.	Propagate *X-Correlation-Id* in all REST clients and messages and generate it in the gateway.	Backend leader	Low	In progress (booking, operations, and payment already forward the header)	TD-007
AP-10	Lack of metrics to check the response time (RNF-001).	*micrometer-registry-prometheus* in the Java services and equivalent metrics in .NET.	Backend leader	Low	Open	TD-012
AP-11	Brute-force attacks or API abuse if the system is exposed.	Request rate limit in the gateway or in Caddy (TD-022); account lockout is already active (AC-25).	Backend leader	Medium	Open	Security policy
AP-12	Components with known vulnerabilities.	Quarterly review with *./mvnw versions:display-dependency-updates*, *dotnet list package --vulnerable*, and *npm audit*.	Backend and frontend leaders	Medium	In progress (permanent)	OWASP A06
AP-13	Confusion caused by the own numbering of the payment changelogs, which repeats the global numbers.	Document the rule in the migration strategy and use the *pay-* prefix in the identifiers.	Backend leader	Low	Closed (documented)	TD-019
AP-14	Features built that contradict decisions already recorded (cause of AC-11 and AC-12).	Review the current ADRs and the navigation map before starting a feature, and consult the team when there is a contradiction.	Whole team	Medium	In progress (permanent)	Lesson of ADR-015

Note. Prepared by the authors from the risk register, the technical backlog, and the security policy of the project.

12. Register of Improvement Actions

Table 9

Improvement Actions

ID	Improvement opportunity	Expected benefit	Responsible	Priority	Status	Origin
AM-01	Automatic operator assignment when a booking is confirmed, reusing *AssignmentPolicy*.	The administrator stops assigning every booking by hand (RF-008).	Backend leader	Medium	Closed (2026-10-16): *BookingConfirmedListener* and *AutoAssignmentPolicy*, with the optional choice of the customer; test *booking-flow.test.js*	TD-020, ADR-014

ID	Improvement opportunity	Expected benefit	Responsible	Priority	Status	Origin
AM-02	Apply the permission matrix of *security.role_permission* to the real authorization of each endpoint.	Granular access control configurable from the panel.	Backend leader	Low	Open (requires an ADR)	ADR-015
AM-03	Rescheduling of bookings by the customer from the web and the app (the API already offers *PUT /api/v1/bookings/{id}*).	The customer does not have to cancel and book again.	Frontend leader	Medium	Open	User Manual
AM-04	Complete the mobile application and build a version with EAS for push notifications.	Immediate notices on the phone.	Frontend leader	Medium	Closed (2026-10-16): APK built with EAS and published at */app/* on the server, with push notices of new versions	ADR-011
AM-05	RS256 signing and refresh tokens.	Lower impact of a secret leak and longer sessions without logging in again.	Backend leader	Low	Open	ADR-006
AM-06	Store receipts, QR codes, and photos outside the database.	Lighter database and faster backups.	Backend leader	Low	Open	Recommended evolution
AM-07	Separate the display fields and the real discount fields in *promotion.promotion*.	Prevents the displayed package and the applied discount from contradicting each other.	Backend leader	Low	Open	ADR-015
AM-08	Publish the built web application behind the gateway with HTTPS.	Production-ready environment, without a development server.	Frontend leader	Medium	Closed (2026-10-16): web container with nginx behind Caddy and HTTPS on the public server (Deployment Document)	Implementation Procedure, R-I08
AM-09	Coverage (JaCoCo) and static analysis (SonarQube) tools.	Objective code quality metrics.	Project leader	Low	Open	Testing strategy
AM-10	Automated end-to-end test across services (registration → booking → notification → payment).	Early detection of integration failures.	Backend leader	Medium	Closed (2026-10-16): *lavarapido-infra/e2e*, 19 tests passed against the containers	Testing strategy
AM-11	Replace the figure placeholders of the manuals with real screenshots of the final version.	Complete manuals according to the SENA guide.	Frontend leader	High	Closed (2026-10-16): manuals 4.0 with real screenshots of the system	Annex A of each manual
AM-12	Use the execution statuses *PAUSED* and *WITH_ISSUE* in the operator interface.	Record pauses and issues of the service.	Frontend leader	Low	Open	Data model
AM-13	Expose in the panel the sending of general announcements (*SYSTEM_MESSAGE*) to groups of users.	Direct communication with customers and operators without using the API.	Frontend leader	Low	Open	Administrator Manual

Note. Prepared by the authors from the recommended evolution of the Technical Manual, the ADRs, and the limitations documented in the manuals.

13. Follow-Up Indicators

Table 10

Action Management Indicators

Indicator	Formula	Target	Value as of 2026-10-16
Closed corrective actions	Closed corrective actions / recorded corrective actions × 100	≥ 70% per delivery	39 / 41 = 95%
Open P0 and P1 corrective actions	Number of ACO with severity P0 or P1	0 before an implementation	0
On-time closure	Actions closed within the time of their severity / closed actions × 100	≥ 90%	To be measured from version 1.0
Recurrence	Closed corrective actions whose problem reappears / closed corrective actions × 100	0%	0%
Escaped defects	Defects found in acceptance or operation / total defects of the period × 100	Downward trend	To be measured from the first implementation
Automated tests	Tests run and passed in every repository	Increasing	639 unit and integration tests (web 43, gateway 34, Java services 421, and payment-service 141) and 19 end-to-end tests

Note. Prepared by the authors. The test count corresponds to the run of October 16, 2026; version 1.0 counted annotations (about 400).

The project leader reviews the indicators at each retrospective and before each implementation. With AC-25 and AC-26 closed there are no open P0 or P1 corrective actions left, so entry criterion E6 of the Implementation Procedure is met. The two open corrective actions (ACO-04 and ACO-13) are P2 and P3 and have a recorded action.

14. Lessons Learned

Table 11

Lessons Learned

No.	Lesson	Related actions
L-01	Storing time in UTC and converting it only at the edges prevents time shifts.	AC-01
L-02	A service must not start before its dependencies are healthy; the orchestration must guarantee it.	AC-02
L-03	Agreeing on the data model before coding a service prevents duplicate implementations.	AC-03
L-04	Every decision already recorded in an ADR must be reviewed before building; a contradiction is discussed before coding.	AC-11, AC-12, AP-14
L-05	A business rule must live in a single service, and the screens must query it, not repeat it.	AC-15
L-06	A preference that the user changes must be saved in their account to have an effect.	AC-16, AC-17
L-07	A new service is not finished until it has a container, a gateway route, Compose variables, and published events.	AC-04, AC-05, AC-06
L-08	Administrative endpoints also need strict input validation and allowlists.	AC-18, AC-19
L-09	Documentation is updated in the same pull request as the code; otherwise, it becomes outdated within days.	AC-23, AP-05
L-10	An accounting ledger needs reversal movements and concurrency control from day one.	AC-09
L-11	A "create if it does not exist" operation must handle the race between two simultaneous requests.	AC-35
L-12	Each interface option must say exactly what it does with the person's data: deactivating is not deleting.	AC-37

Note. Prepared by the authors from the register of closed risks and the actions of this document.

15. Glossary

Table 12

Glossary

Term	Definition
ADR	Architecture decision record.
Correction	Immediate action that eliminates the symptom of a nonconformity without addressing its cause.
Corrective action	Action that eliminates the cause of a nonconformity that occurred, to prevent its recurrence.
Effectiveness	Extent to which an action achieves the planned result.
Improvement action	Action that increases the value or quality of the software without an existing failure.
Ledger	Accounting record of point movements, whose balance is that of the last movement.
Nonconformity	Failure to meet a requirement, a recorded decision, or what is documented.
PDCA	Plan, do, check, and act cycle of continual improvement.
Post-mortem	Blameless analysis of a serious incident, with its cause and its actions.
Preventive action	Action that eliminates the cause of a potential nonconformity.
Root cause	Fundamental cause of a problem on which action can be taken so that it does not recur.
Technical debt	Pending work that makes the software work, but not as it should.

Note. Prepared by the authors from ISO (2015), ISO et al. (2022), and the project documentation.

References

Congreso de la República de Colombia. (2012). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales* [Statutory Law 1581 of 2012, which issues general provisions for the protection of personal data]. http://www.secretariasenado.gov.co/senado/basedoc/ley_1581_2012.html

International Organization for Standardization. (2015). *Quality management systems — Requirements* (ISO 9001:2015). https://www.iso.org/standard/62085.html

International Organization for Standardization, International Electrotechnical Commission, & Institute of Electrical and Electronics Engineers. (2017). *Systems and software engineering — Software life cycle processes* (ISO/IEC/IEEE 12207:2017). https://www.iso.org/standard/63712.html

International Organization for Standardization, International Electrotechnical Commission, & Institute of Electrical and Electronics Engineers. (2022). *Software engineering — Software life cycle processes — Maintenance* (ISO/IEC/IEEE 14764:2022). https://www.iso.org/standard/80710.html

Servicio Nacional de Aprendizaje. (2026). *Guía para elaborar manuales de software: Manual de usuario, manual técnico, manual de instalación y manual de administrador* [Guide for writing software manuals: User manual, technical manual, installation manual, and administrator manual] [Unpublished instructional material]. Technologist in Software Analysis and Development program.

Vehicle Washing Development Team. (2026a). *Vehicle washing management system: Project documentation* (vehicle-w-docs) [Unpublished project documentation]. GitHub. https://github.com/code-sena/vehicle-w-docs

16. Annexes

Annex A. Action Record Form

Table 13

Action Record Form

Field	Content
ID	AC-NN, ACO-NN, AP-NN, or AM-NN
Type	Corrective / Preventive / Improvement
Recording date and who detected it	
Description of the nonconformity, risk, or opportunity	
Initial evidence (log, screenshot, test, error code)	
Severity (P0 to P3) and priority (High, Medium, Low)	
Immediate correction applied (if any)	
Root cause and technique used	
Planned action, person responsible, and committed date	
Branch, pull request, and related ADR	
Closure criterion	
Effectiveness verification (who, when, evidence)	
Status and closure date	
Lesson learned	

Annex B. Root Cause Analysis Example

Table 14

Five Whys of Action AC-01 (Times Shifted by Five Hours)

Question	Answer
Why did the customer see their booking at another time?	Because the time shown differed by five hours from the one chosen.
Why did it differ?	Because the time was stored converted with one time zone and read with another.
Why were different time zones used?	Because the JVM of the service took the time zone of the machine where it ran.
Why did the machine's time zone affect the data?	Because the code converted dates without an explicit time zone.
Why was there no explicit time zone?	Because there was no team rule for handling time.
Root cause	Absence of a time convention: "store in UTC and convert only at the edges with *BOOKING_ZONE*" was adopted, and the JVM runs in UTC.

Note. Prepared by the authors from the closed risk C-001 of the project.

Annex C. Traceability Between Actions, Risks, and Technical Debt

Table 15

Traceability

Project risk	Technical debt	Actions
R-002 Partial payment-service (closed as C-005)	TD-014, TD-015, TD-016, TD-017, TD-018	AC-04 to AC-10, AC-27
R-003 Regressions without a safety net	TD-001, TD-006, TD-010	AC-24, AP-01, AP-07, AM-10
R-004 Secret leakage	—	AP-03
R-005 Lost event	TD-011	AP-02
R-007 Known requirement gaps (closed as C-006)	TD-002, TD-003, TD-009	AC-16, AC-17, AC-25, AC-31
R-008 Data loss	—	AP-04

Project risk	Technical debt	Actions
R-010 Documentation drift	—	AC-23, AC-34, AC-38, AP-05
R-011 Public server off	—	Implementation Procedure, R-I04
API abuse	TD-022	AP-11
R-001 operations-service integration (closed as C-004)	TD-020	AM-01
Law 1581 of 2012	—	AC-26, AC-37

Note. Prepared by the authors from the risk register and the technical backlog of the project.